

PRACTICAL SESSION 5

TASK 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

UW PICO 5.09

File: producer_consumer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define NUM_MESSAGES 100000
#define MESSAGE_SIZE 64

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // Child process - Consumer
        close(fd[1]);

        char buffer[MESSAGE_SIZE];
        int count = 0;
        ssize_t bytes_read;
        long total_bytes = 0;

        while ((bytes_read = read(fd[0], buffer, MESSAGE_SIZE)) > 0) {
            total_bytes += bytes_read;
            count++;
        }

        close(fd[0]);
    }
```

```

    printf("Consumer: Received %d messages\n", count);
    printf("Consumer: Total bytes received = %ld\n", total_bytes);

    exit(0);
}
else {
    // Parent process - Producer
    close(fd[0]);

    char message[MESSAGE_SIZE] = "Data generated by producer process";

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < NUM_MESSAGES; i++) {
        if (write(fd[1], message, MESSAGE_SIZE) == -1) {
            perror("write");
            close(fd[1]);
            return 1;
        }
    }

    close(fd[1]);
}

```

UW PICO 5.09

File: producer_consumer.c

```

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    long total_bytes = (long)NUM_MESSAGES * MESSAGE_SIZE;

    printf("Producer: Sent %d messages\n", NUM_MESSAGES);
    printf("Producer: Total bytes sent = %ld\n", total_bytes);
    printf("Communication time = %.6f seconds\n", elapsed);

    if (elapsed > 0) {
        double throughput =
            (total_bytes / (1024.0 * 1024.0)) / elapsed;

        printf("Communication throughput = %.2f MB/s\n",
            throughput);
    }

    return 0;
}

```

```

(base) saiabburi@Geyasree-MacBook-Air Practical5 % clang producer_consumer.c -o producer_consumer
(base) saiabburi@Geyasree-MacBook-Air Practical5 % ls
producer_consumer      producer_consumer.c
(base) saiabburi@Geyasree-MacBook-Air Practical5 % ./producer_consumer
Consumer: Received 100000 messages
Consumer: Total bytes received = 6400000
Producer: Sent 100000 messages
Producer: Total bytes sent = 6400000
Communication time = 0.065491 seconds
Communication throughput = 93.20 MB/s
(base) saiabburi@Geyasree-MacBook-Air Practical5 % pipe()
fork()
write()
read()
wait()
function function function function function>

```

TASK 2: Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

UW PICO 5.09

File: ls_grep.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    // First child: executes ls -l
    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        close(fd[0]);

        // Redirect standard output to pipe
        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", "-l", (char *)NULL);

        perror("execlp ls");
        exit(1);
    }

    // Second child: executes grep ".c"
    pid_t pid2 = fork();

    if (pid2 < 0) {
        perror("fork");
        return 1;
    }
}

```

```

        close(fd[1]);

        // Redirect standard input from pipe
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("grep", "grep", ".c", (char *)NULL);

        perror("execlp grep");
        exit(1);
    }

    // Parent does not use the pipe
    close(fd[0]);
    close(fd[1]);

    // Wait for both child processes
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}

```

```

(base) saiabburi@Geyasree-MacBook-Air Practical5 % nano ls_grep.c
(base) saiabburi@Geyasree-MacBook-Air Practical5 % clang ls_grep.c -o ls_grep
(base) saiabburi@Geyasree-MacBook-Air Practical5 % ls -l
total 160
-rwxr-xr-x  1 saiabburi  staff  33844 Aug 24 09:12 ls_grep
-rw-r--r--  1 saiabburi  staff   1140 Aug 24 09:12 ls_grep.c
-rwxr-xr-x  1 saiabburi  staff  33998 Aug 24 08:56 producer_consumer
-rw-r--r--  1 saiabburi  staff   2035 Aug 24 08:56 producer_consumer.c
(base) saiabburi@Geyasree-MacBook-Air Practical5 % ./ls_grep
-rw-r--r--  1 saiabburi  staff   1140 Aug 24 09:12 ls_grep.c
-rwxr-xr-x  1 saiabburi  staff  33998 Aug 24 08:56 producer_consumer
-rw-r--r--  1 saiabburi  staff   2035 Aug 24 08:56 producer_consumer.c
(base) saiabburi@Geyasree-MacBook-Air Practical5 % ls -l | grep ".c"
-rw-r--r--  1 saiabburi  staff   1140 Aug 24 09:12 ls_grep.c
-rwxr-xr-x  1 saiabburi  staff  33998 Aug 24 08:56 producer_consumer
-rw-r--r--  1 saiabburi  staff   2035 Aug 24 08:56 producer_consumer.c
(base) saiabburi@Geyasree-MacBook-Air Practical5 % ./ls_grep
-rw-r--r--  1 saiabburi  staff   1140 Aug 24 09:12 ls_grep.c
-rwxr-xr-x  1 saiabburi  staff  33998 Aug 24 08:56 producer_consumer
-rw-r--r--  1 saiabburi  staff   2035 Aug 24 08:56 producer_consumer.c

```

